

From Jarvis Rocket Sled to Luke Control Room

A five-page-or-less timeline of the scaffolding, Jarvis legacy, and current Luke build. Honest version: this thing started as a high-speed trading assistant with a toolbelt, then got rebuilt into a local companion shell with trading locked inside a padded, human-gated room.

192

Luke commits inspected

Current repo history spans Apr 19 to May 7. It includes the original Jarvis arc plus the heavier Luke rebuild.

88

Jarvis commits inspected

Jarvis branch stops at Apr 24 and is dirty on `rename/jarvis-to-luke`. Translation: the old lab is still full of labeled jars.

921

Latest Luke tests passed

Full Vitest suite passed after Radar landed: 147 files, 921 passed, 1 skipped. The robot has receipts.

EXECUTIVE TAKE

Jarvis was the prototype organism. It had agents, workflows, trading automation, health logging, research, sweeper/architect scaffolding, and plenty of state files living at repo root like everyone had just survived a hackathon in a broom closet.

Luke is the consolidation project. The current repo adds explicit safety boundaries, a front-facing shell, proof scripts, state path management, a brain dashboard, Daily, Operator V2, Katbot, Ninja bridge work, and now Radar. The center of gravity moved from "can it do the thing?" to "can it do the thing without quietly sawing through the floor?"

Scaffolding matured. Early scaffolding was Agent 09 Architect, Agent 10 Sweeper, Agent 13 Workflows, suggestions, SWEEPER maps, and root logs. Current scaffolding is `agent` goal plans, proof runners, docs/source-of-truth files, ignored runtime state, and tests that enforce the UI and trading boundaries.

Presentation line: "Jarvis proved the engine. Luke is the chassis, dashboard, guardrails, and yes, the cup holder we apparently needed."

Evidence base: `C:\Users\conor\luke`, `C:\Users\conor\jarvis`, git logs, folder inventories, roadmap/docs, package scripts, and recent test/proof results.

TIMELINE

Apr 19	Architect starts the repo. Initial commit is `arch: initial commit - git init by agent-09`. Scaffold is literal: Architect, Sweeper, workflows, SWEEPER maps, proposals, and raw state. This is the "let's build the builder" phase.
Apr 20	Jarvis v1 becomes usable. Commit says "Jarvis v1 - built April 20 2026 - because I'm tired of losing." Subtle? No. Effective? Apparently yes. Bracket calculator, popup chart, safe layout, audits, and production-readiness claims appear quickly.
Apr 21	Trading and UI sprint explodes. 38 commits. Smart paste routing, tooltips, `/luke`, dry-fire, layout, `/balance`, parsers, Kat ingestion, confluence, session replay, runner protocol, Apex floor checks. Jarvis becomes a trading cockpit with too many switches but real momentum.
Apr 22	Audit/hardening day. 34 commits. Security fixes, cost controls, prompt caching, pruning, prompt/routing fixes, Bobby/Kat/Ximes confluence, readiness warnings. This is where the project starts admitting: "maybe the stove should not be hot by default."
Apr 23-24	Eval build and rename plan. Saty parser, Massive API, image routing, Kat fixes, SPX price endpoint, vision error surfacing. Jarvis branch shows `CONOR_RENAME_CHECKLIST.md`, old Jarvis files deleted, Luke files added, and many dirty rename edits. The rename is real but not cleanly closed in that folder.
Apr 27-28	Luke stabilization begins in earnest. The current Luke repo bridges legacy confluence, hardens JSON reads, removes memorial-era care surfaces, normalizes slash-command text, retires dead premarket UI, and documents operator workflow. Sentiment gets converted into system boundaries. Good move.
Apr 30	Repo hygiene and compatibility work. Final handoff triage docs, Saty auto-pull, SDK wrapper, heatmap tests, ingestion extraction, and archived/deferred work cleanup. This is the first obvious "we need a maintainable machine, not an archeological dig" phase.
May 2-5	Luke gets its shell and proof culture. Decision spine, runtime paths, staged flow proof, historical replay, brain shell spines, agent scaffold, launcher cleanup, Operator V2, Daily, UI proof, current status docs, startup gates, and live execution blocks. The dashboard stops being just plumbing and becomes the front door.
May 6-7	Trading scope narrows, Radar lands. Ximes is retired from the trading surface, Katbot Stage 2/routing hardens, and Radar becomes a front-shell intake/synthesis panel. No duplicate visible launch paths remain in the shell after the latest cleanup.

Commit density tells the story: Jarvis had the violent birth. Luke had the cleanup surgery.

FOLDER MAP: WHAT IS INVOLVED

AREA	JARVIS EVIDENCE	LUKE EVIDENCE	MEANING
Agents	14 agent files, including trader, health, finance, research, architect, sweeper, workflows, Kat.	15 agent files; adds `agent-00-brain`, fallback growth, Ninja watchdog, larger Kat agent.	Jarvis had task agents. Luke adds an orchestrating brain layer and more explicit sidecars.
Scaffold	`agents/agent-09-architect.js`, `agent-10-sweeper.js`, `SWEEPER_MAP.json`, proposals, suggestions.	`agent/PLANS.md`, `agent/goals/...`, proof scripts, docs, reports, paths-managed state.	Scaffolding moved from "agents mutate/advice" to "plans, gates, tests, proof, then commit." Less magic wand, more clipboard.
UI	`chat.html` does a lot: chat, panels, tooltips, staged execution buttons, Luke panel.	`luke-shell.html`, `daily-window.html`, `operator-v2.html`, `trading-window.html`, `brain-dashboard.html`, `radar-dashboard.html`.	Luke splits surfaces by job. This reduces accidental cross-wiring and duplicate paths.
State	Many root JSON/JSONL files: logs, memory, boot checks, tool health, action logs.	`state/events`, `state/snapshots`, `data`, `artifacts`, `reports`, `lib/paths.js` registry.	State is being domesticated. Still not spotless, but no longer sleeping on the couch at repo root.
Trading	`trading/` with live/paper/shadow execution and router; strong Apex focus.	`trading/` plus explicit proof scripts, read-only UI, live/staged gates, Operator V2.	Trading remains core, but it is now a contained module instead of the whole personality.
Workflows	Tradovate YAML workflows and workflow recordings.	Tradovate YAML still exists, with runtime recordings moved under state runtime.	Automation survived, but Luke is forcing it into safer storage and proof lanes.
Research	`research-agent.py`, suggestions, strategy docs, Discord exports.	Katbot Stage 2, research scripts, fake-breakdown, ladder reclaim, Pine, Radar.	Research has grown from article digestion into a source-intake and replay/review system.

CURRENT EVIDENCE SNAPSHOT

Luke current repo

- Branch: `refactor/decision-spine-cleanup`, ahead of origin by 25 commits.
- Top active folders include `agents`, `lib`, `scripts`, `tests`, `docs`, `trading`, `tradingview`, `reports`, `agent`, and `proposals`.
- Package scripts expanded from basic start/test to proof, runtime, research, replay, Ninja, Katbot, and UI verification commands.

Jarvis legacy repo

- Branch: `rename/jarvis-to-luke`, dirty with rename edits and untracked Luke files.
- Still contains root logs/state and old rename checklist residue.
- Useful as evidence of origin, not as the clean current source of truth.

The folder story is simple: Jarvis kept everything close. Luke is moving it into lanes.

WHAT CHANGED ARCHITECTURALLY

1. From single cockpit to front shell

Jarvis mostly lived through `chat.html`. Luke now has `/shell` as the front door, with drilldowns for Daily, Radar, trading, brain status, Operator V2, and replay windows. The recent Radar work also removed duplicate visible launch paths in the front UI.

2. From execution confidence to safety proof

Jarvis had human gates, but the UI still carried live/staged execution affordances. Luke increasingly says: read-only, replay, proof first. Current docs explicitly block live readiness without separate market data and broker proof.

3. From root-state soup to registered paths

Jarvis root included many JSON/JSONL runtime files. Luke uses `lib/paths.js`, `state/events`, `state/snapshots`, `data`, and ignored artifacts. Not glamorous, but neither is labeling the breaker box. You still do it.

4. From scaffold agents to goal protocol

Architect/Sweeper were early self-maintenance scaffolds. Luke now adds `agent` plans, proof scripts, regression tests, and source-of-truth docs. The scaffolding is less "AI, please freestyle" and more "AI, here is the checklist; try not to invent a basement."

5. From trading-only to companion lanes

Daily, History/Career, Automation Business, Developer Stack, Luke System, and Radar show the broader companion direction. Trading is still important, but it has roommates now, and some of them pay rent.

6. From source capture to synthesis

Jarvis had Discord, Kat, Bobby, Ximes, research, and manual paste workflows. Luke adds Radar as the front-facing intake layer: capture raw sources, preserve provenance, classify, summarize, and feed review queues without claiming magic trading certainty.

The honest joke: Jarvis was "what if we bolted a jet engine to a spreadsheet?" Luke is "what if the jet engine had a dashboard, brakes, and a person with veto power?"

RESIDUAL RISKS

- **Jarvis is not clean.** It is useful historical evidence, but it is dirty mid-rename. Do not treat it as a deploy target.
- **Luke still carries residue.** Archives, reports, proof artifacts, Pine variants, and generated evidence are present. Some are useful. Some are probably digital barnacles.
- **Live trading remains blocked by design.** Current status still calls out missing live/delayed ES 1m OHLC proof, TradingView compile checks, and explicit broker/live unlock gates.
- **Docs can drift.** `docs/CURRENT\_STATUS.md` was May 5, while Radar landed May 7. The code is ahead of some docs. Normal, but worth updating before any formal external handoff.

This is a strong system, not a finished statue. Statues do not run tests. Luke does.

PRESENTATION READOUT

The story in one minute

Jarvis began as a fast, practical assistant for trading, research, and life operations. It quickly grew agents, workflows, confluence parsing, live/paper/shadow trading paths, and a lot of root-level state. Then the project hit the obvious wall: if this thing is going to matter, it cannot be a pile of clever scripts with a steering wheel taped on. Luke is the consolidation: local front shell, separated surfaces, explicit proof scripts, read-only default trading, organized state, and companion lanes like Daily and Radar. The project is not smaller now. It is safer, more presentable, and less likely to bite the person feeding it.

What I would say confidently

- The current source of truth is `C:\Users\conor\luke`, not `jarvis`.
- Jarvis explains the origin and design DNA.
- Scaffolding evolved from self-mutating agents into goal plans, proof scripts, and tests.
- The front UI is now the Luke shell, with backend dashboards as drilldowns.
- Trading is intentionally human-gated and should stay that way until a separate proof phase says otherwise.

What I would not oversell

- I would not claim Jarvis migration is historically clean. It is not.
- I would not claim all residue is gone. It is not, and some residue is evidence.
- I would not claim live trading readiness. The repo itself says no.
- I would not claim docs are fully current after May 7. Radar is newer than the May 5 status doc.
- I would not trust any single dashboard as the whole truth. Use tests, routes, and git history together.

RECOMMENDED NEXT MOVES

1. **Update `docs/CURRENT\_STATUS.md`** with Radar, the unique front-route cleanup, and the latest test counts.
2. **Declare `jarvis` historical/archive** unless you need a specific file from it. It is not the clean operating base.
3. **Run a residue audit as its own commit series:** reports, artifacts, Pine variants, archive, generated state, and old rename docs.
4. **Keep Radar front-facing but not autonomous:** capture, connect, brief, alert for review. No secret confidence casino.
5. **Do not unlock live execution** until market data, TradingView compile, broker gates, and proof docs all agree.

Bottom line: Luke is no longer just "Jarvis renamed." It is Jarvis after a safety audit, a repo cleanup intervention, a UI identity crisis, and a useful new Radar antenna.